Python Engineering

WEEK 04 - OOP, Files & Testing | CLASS 13 - Classes & Objects | EX21

Goal: introduce object-oriented programming by building a class that represents the state of vertical motion. We learn class, object, __init__, self, attributes, and methods.

Path

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_13_classes_objects/
        |-- notes/
        `-- exercises/
            `-- ex21_vertical_state_class/
                |-- vertical_state.py
                `-- main.py
```

Reference values

Case	Initial values	Reference
Earth object	h=100 m, v=20 m/s, t=0 s, g=9.81	specific energy = 1181.0 J/kg
Moon object	h=100 m, v=20 m/s, t=0 s, g=1.62	specific energy = 362.0 J/kg
Earth after dt=0.1 s	-	h=102.0, v=19.019, t=0.1
Moon after dt=0.1 s	-	h=102.0, v=19.838, t=0.1

BLOCK A - Create the class

```
class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g
```

__init__ configures the initial state of each new object.

BLOCK B - specific_energy()

```
def specific_energy(self):
    return self.g * self.h + 0.5 * self.v**2
```

BLOCK C - state_vector()

```
def state_vector(self):
    return np.array([self.h, self.v, self.t])
```

BLOCK D - is_ascending()

```
def is_ascending(self):
    return self.v > 0.0
```

BLOCK E - euler_step()

```
def euler_step(self, dt):
    self.h = self.h + self.v * dt
    self.v = self.v - self.g * dt
    self.t = self.t + dt
```

This method mutates the object's stored state.

BLOCK F - Create two independent objects

```
earth_state = VerticalState(h=100.0, v=20.0)
moon_state = VerticalState(h=100.0, v=20.0, g=1.62)
```

Both objects belong to the same class but store independent attributes.

BLOCK G - Use methods

```
earth_energy = earth_state.specific_energy()
moon_energy = moon_state.specific_energy()

earth_before = earth_state.state_vector()
moon_before = moon_state.state_vector()

earth_state.euler_step(0.1)
moon_state.euler_step(0.1)

earth_after = earth_state.state_vector()
moon_after = moon_state.state_vector()
```

BLOCK H - main.py

```
from vertical_state import VerticalState

def main():
    # Create Earth and Moon objects
    # Call methods
    # Compare their independent states
    # Print results

if __name__ == '__main__':
    main()
```

FINAL VALIDATION

```
Explain in comments:
- What is a class?
- What is an object or instance?
- What does __init__ do?
- What does self refer to?
- What is the difference between parameter h and self.h?
- What is an attribute?
- What is a method?
- Why can earth_state and moon_state have different g values?
- What changes inside the object when euler_step() is called?
- Why is earth_state.specific_energy() different from VerticalState.specific_energy?
- What advantages does this representation have over passing h, v, t and g to every function?
```

Completion criterion

EX21 is complete when you can create several objects from one class, understand that each object stores its own state, and clearly distinguish class, object, attribute, and method.